

Examen 2024-2025

1. Questions de Cours (20 points) / 1. 课程问题 （20 分）

1) Qu'est-ce que MPI (Message Passing Interface) ? Donnez un exemple d'utilisation simple. (4 points) / 1) 什么是 MPI（消息传递接口）？请给出一个简单的使用示例。（4 分）

Correction / 参考答案

MPI est une bibliothèque permettant la communication entre processus dans un environnement parallèle. / MPI 是一个用于并行环境中进程间通信的库。
Exemple simple : envoyer un message d'un processus à un autre. / 简单示例：从一个进程向另一个进程发送一条消息。

2) Explique ce qu'est un communicator en MPI. Donnez un exemple de création et d'utilisation d'un communicator. (4 points) / 2) 解释 MPI 中的 communicator 是什么，并给出一个创建和使用 communicator 的例子。（4 分）

Correction / 参考答案

Explication d'un communicator en MPI : / MPI 中 communicator 的解释：

- En MPI (Message Passing Interface), un communicator est un objet qui définit un groupe de processus pouvant communiquer entre eux. / 在 MPI 中，communicator 是一个定义可相互通信进程组的对象。
- Il permet de définir des sous-ensembles de processus qui peuvent échanger des messages de manière isolée des autres processus. / 它可以定义进程的子集合，使这些进程能够与其他进程隔离地交换消息。
- Les communicateurs sont essentiels pour structurer les communications dans les applications parallèles. / communicator 对于组织并行程序中的通信非常重要。

Exemple de création et d'utilisation d'un communicator : / communicator 的创建与使用示例：

Voici un exemple simple en C utilisant MPI pour créer et utiliser un communicator. / 下面给出一个用 C 和 MPI 创建并使用 communicator 的简单示例。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int world_rank, world_size;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    // Création d'un nouveau communicateur pour les processus de rang pair
    MPI_Comm new_comm;
    MPI_Comm_split(MPI_COMM_WORLD, world_rank % 2, world_rank, &new_comm);
    int new_rank, new_size;
    MPI_Comm_rank(new_comm, &new_rank);
    MPI_Comm_size(new_comm, &new_size);
    printf("Processus global %d sur %d - Processus local %d sur %d\n",
           world_rank, world_size, new_rank, new_size);
    // Utilisation du nouveau communicateur pour une communication
    if (new_rank == 0) {
        int message = 42;
        MPI_Send(&message, 1, MPI_INT, 1, 0, new_comm);
        printf("Processus %d a envoyé le message %d\n", new_rank, message);
    } else if (new_rank == 1) {
        int message;
        MPI_Recv(&message, 1, MPI_INT, 0, 0, new_comm, MPI_STATUS_IGNORE);
        printf("Processus %d a reçu le message %d\n", new_rank, message);
    }
    MPI_Comm_free(&new_comm);
    MPI_Finalize();
    return 0;
}
```

Explication du code : / 代码说明：

- Initialisation de MPI : `MPI_Init` initialise l'environnement MPI. / MPI 初始化: `MPI_Init` 用于初始化 MPI 环境。
- Obtention du rang et de la taille du communicateur global : `MPI_Comm_rank` et `MPI_Comm_size` donnent les informations de `MPI_COMM_WORLD`. / 获取全局 communicator 的秩与大小: `MPI_Comm_rank` 和 `MPI_Comm_size` 给出 `MPI_COMM_WORLD` 的信息。
- Création d'un nouveau communicateur : `MPI_Comm_split` divise `MPI_COMM_WORLD` en deux groupes (rangs pairs / impairs). / 创建新的 communicator: `MPI_Comm_split` 将 `MPI_COMM_WORLD` 划分为两个组（偶数秩 / 奇数秩）。
- Obtention du rang et de la taille du nouveau communicateur : `MPI_Comm_rank` et `MPI_Comm_size` sont appliqués au nouveau communicateur. / 获取新 communicator 的秩与大小: 对新 communicator 调用 `MPI_Comm_rank` 和 `MPI_Comm_size`。
- Communication à l'intérieur du nouveau communicateur : les processus de rang local 0 et 1 échangent un message. / 在新 communicator 内进行通信: 本地秩为 0 和 1 的进程交换消息。
- Libération du communicateur : `MPI_Comm_free` libère le nouveau communicateur. / 释放 communicator: `MPI_Comm_free` 释放新 communicator。
- Finalisation : `MPI_Finalize` termine l'environnement MPI. / 结束: `MPI_Finalize` 终止 MPI 环境。

Conclusion : / 结论：

- Les communicateurs MPI permettent de structurer les communications entre processus de manière flexible et efficace. / MPI communicator 能够以灵活且高效的方式组织进程间通信。
- Ils sont essentiels pour des applications parallèles complexes où différents groupes de processus communiquent de manière isolée. / 在复杂并行程序中，不同进程组需要彼此隔离通信时，它们是必不可少的。

3) Quelle est la différence entre les communications bloquantes et non-bloquantes en MPI ? Donnez un exemple de chaque type de communication. (4 points) / 3) MPI 中阻塞通信与非阻塞通信有什么区别？请分别给出一个例子。（4 分）

Correction / 参考答案

Communications bloquantes : / 阻塞通信：

- Dans une communication bloquante, l'appel de la fonction ne retourne pas tant que l'opération n'est pas terminée. / 在阻塞通信中，函数调用在操作完成前不会返回。
- Le processus appelant reste bloqué jusqu'à la fin de la communication. / 调用进程会一直阻塞到通信结束。

Exemples / 示例：

```
MPI_Send(buffer, count, datatype, dest, tag, comm);
MPI_Recv(buffer, count, datatype, source, tag, comm, status);
```

Communications non-bloquantes : / 非阻塞通信：

- Dans une communication non-bloquante, l'appel retourne immédiatement et le processus peut continuer son exécution. / 在非阻塞通信中，调用会立即返回，进程可以继续执行。
- L'opération se fait en arrière-plan et doit être finalisée explicitement. / 通信操作在后台进行，并且需要显式完成。

Exemples / 示例：

```
MPI_Isend(buffer, count, datatype, dest, tag, comm, request);
MPI_Irecv(buffer, count, datatype, source, tag, comm, request);
```

Pour compléter ces opérations, on utilise `MPI_Wait` ou `MPI_Test`. / 要完成这些操作，需要使用 `MPI_Wait` 或 `MPI_Test`。

4) Communications collectives (8 points) / 4) 集合通信 （8 分）

4.1 Analysez le code suivant utilisant `MPI_Bcast` et expliquez son fonctionnement / 4.1 分析下面使用 `MPI_Bcast` 的代码，并解释其工作原理

```
int main(int argc, char *argv[]) {
    int rank, size, data;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (rank == 0) {
        data = 42;
    }
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received data: %d\n", rank, data);
    MPI_Finalize();
    return 0;
}
```

Correction / 参考答案

`MPI_Bcast` diffuse la valeur de `data` du processus 0 vers tous les autres processus. / `MPI_Bcast` 会把进程 0 中 `data` 的值广播给所有其他进程。

4.2 Analysez le code suivant utilisant `MPI_Reduce` et expliquez son fonctionnement / 4.2 分析下面使用 `MPI_Reduce` 的代码，并解释其工作原理

```
int main(int argc, char *argv[]) {
    int rank, size, local_sum = rank, global_sum;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        printf("Global sum: %d\n", global_sum);
    }
    MPI_Finalize();
    return 0;
}
```

Correction / 参考答案

`MPI_Reduce` additionne les valeurs de `local_sum` de tous les processus et stocke le résultat dans `global_sum` du processus 0. / `MPI_Reduce` 会把所有进程中的 `local_sum` 相加，并将结果存入进程 0 的 `global_sum` 中。

5) Comment MPI gère-t-il la synchronisation entre les processus ? Donnez un exemple de synchronisation explicite et implicite. (4 points) / 5) MPI 如何处理进程之间的同步？请分别给出显式同步和隐式同步的例子。（4 分）

Correction / 参考答案

MPI utilise des barrières (`MPI_Barrier`) pour la synchronisation explicite. / MPI 使用屏障（`MPI_Barrier`）进行显式同步。

Les communications collectives comme `MPI_Bcast` impliquent une synchronisation implicite. / 像 `MPI_Bcast` 这样的集合通信则包含隐式同步。

2. Exercices (30 points) / 2. 练习题（30 分）

1) Écrire un programme MPI en C ou C++ qui calcule la somme des éléments d'un tableau de taille n en utilisant p processus. Utilisez une communication collective pour réduire les résultats. (10 points) / 1) 用 C 或 C++ 编写一个 MPI 程序，使用 p 个进程计算大小为 n 的数组元素之和。请使用集合通信归并结果。（10 分）

Correction / 参考答案

Utilisez `MPI_Reduce` pour additionner les sommes partielles calculées par chaque processus. / 使用 `MPI_Reduce` 对各进程计算出的局部和进行求和。

2) Implémentez un programme MPI en C ou C++ qui trouve le maximum d'un tableau de taille n en utilisant p processus. Utilisez une communication collective pour obtenir le résultat final. (10 points) / 2) 实现一个 C 或 C++ 的 MPI 程序，使用 p 个进程求大小为 n 的数组最大值。请使用集合通信得到最终结果。（10 分）

Correction / 参考答案

Utilisez `MPI_Reduce` avec l'opérateur `MPI_MAX` pour trouver le maximum global. / 使用带有 `MPI_MAX` 操作符的 `MPI_Reduce` 来求全局最大值。

3) Correction de code (20 points) / 3) 代码改错（20 分）

Cas 1 / 情况 1

Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées : / 请修改下面的代码，使其能在任意数量的 MPI 进程下正确运行。说明错误及修正内容：

```
int main(int argc, char *argv[]) {
    int rank, size, data = rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Allreduce(&data, &data, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    printf("Process %d has data: %d\n", rank, data);
    MPI_Finalize();
    return 0;
}
```

Correction : / 参考答案：

- Erreur : `data` est utilisé à la fois comme buffer d'envoi et de réception. / 错误： `data` 同时被当作发送缓冲区和接收缓冲区使用。
- Correction : utiliser deux variables distinctes. / 修正：使用两个不同的变量。

```
int main(int argc, char *argv[]) {
    int rank, size;
    int send_data, recv_data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    send_data = rank;
    MPI_Allreduce(&send_data, &recv_data, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    printf("Process %d has data: %d\n", rank, recv_data);

    MPI_Finalize();
    return 0;
}
```

Cas 2 / 情况 2

Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées : / 请修改下面的代码，使其能在任意数量的 MPI 进程下正确运行。说明错误及修正内容：

```
int main(int argc, char *argv[]) {
    int rank, size, local_data = rank, gathered_data[10];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Gather(&local_data, 1, MPI_INT, gathered_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            printf("Gathered data[%d]: %d\n", i, gathered_data[i]);
        }
    }
    MPI_Finalize();
    return 0;
}
```

Correction : / 参考答案：

- Erreur : `gathered_data` doit être de taille `size`. / 错误： `gathered_data` 的大小应为 `size`。
- Correction : allouer dynamiquement `gathered_data` en fonction de `size`. / 修正：根据 `size` 动态分配 `gathered_data`。

```
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int rank, size, local_data;
    int *gathered_data = NULL;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    local_data = rank;
    if (rank == 0) {
        gathered_data = (int *)malloc(size * sizeof(int));
    }

    MPI_Gather(&local_data, 1, MPI_INT, gathered_data, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            printf("Gathered data[%d]: %d\n", i, gathered_data[i]);
        }
        free(gathered_data);
    }

    MPI_Finalize();
    return 0;
}
```

Cas 3 / 情况 3

Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées : / 请修改下面的代码，使其能在任意数量的 MPI 进程下正确运行。说明错误及修正内容：

```
int main(int argc, char *argv[]) {
    int rank, size, data;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (rank == 0) {
        data = 42;
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
    }
    printf("Process %d received data: %d\n", rank, data);
}
```

Correction : / 参考答案：

- Erreur : seul le processus 1 reçoit les données. / 错误：只有进程 1 接收到了数据。
- Correction : utiliser une boucle pour envoyer les données à tous les processus. / 修正：使用循环把数据发送给所有进程。

```
int main(int argc, char *argv[]) {
    int rank, size, data = 0;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        data = 42;
        for (int dest = 1; dest < size; dest++) {
            MPI_Send(&data, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
        }
    } else {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
    }

    printf("Process %d received data: %d\n", rank, data);

    MPI_Finalize();
    return 0;
}
```

Cas 4 / 情况 4

Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées : / 请修改下面的代码，使其能在任意数量的 MPI 进程下正确运行。说明错误及修正内容：

```
int main(int argc, char *argv[]) {
    int rank, size, data_in, data_out;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    data_in = rank;
    MPI_Send(&data_in, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(&data_out, 1, MPI_INT, size - 1, 0, MPI_COMM_WORLD, &status);
    MPI_Finalize();
    return 0;
}
```

Correction : / 参考答案：

Erreur : Il y a un deadlock lié à l'utilisation de communications MPI bloquantes. / 错误：由于使用阻塞式 MPI 通信，会产生死锁。
Tous les processus peuvent se bloquer lors de la phase d'envoi. / 所有进程都可能在发送阶段阻塞。

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size, data_in, data_out;
    MPI_Status status;
    MPI_Request request_send, request_recv;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    data_in = rank;

    // Calculer le rang du processus suivant et précédent
    int next_rank = (rank + 1) % size;
    int prev_rank = (rank - 1 + size) % size;

    // Envoyer un message au processus suivant (non bloquant)
    MPI_Isend(&data_in, 1, MPI_INT, next_rank, 0, MPI_COMM_WORLD, &request_send);
    // Recevoir un message du processus précédent (non bloquant)
    MPI_Irecv(&data_out, 1, MPI_INT, prev_rank, 0, MPI_COMM_WORLD, &request_recv);

    // Attendre la fin des communications
    MPI_Wait(&request_send, MPI_STATUS_IGNORE);
    MPI_Wait(&request_recv, &status);

    // Afficher les résultats
    printf("Processus %d a reçu %d du processus %d\n", rank, data_out, prev_rank);

    MPI_Finalize();
    return 0;
}
```

3. Problème (50 points) / 3. 综合题 (50 分)

Contexte / 背景

Implémentez un programme qui distribue un tableau d'entiers entre plusieurs processus, calcule la moyenne de ces éléments de manière parallèle, et envoie les résultats au processus principal pour calculer la moyenne totale. / 实现一个程序，将一个整数数组分发到多个进程中，并行计算这些元素的平均值，再把结果发送给主进程以计算总体平均值。

Vous disposez de p processus. / 共有 p 个进程可用。

Questions / 问题

1) Analyse du problème (10 points) / 1) 问题分析 (10 分)

Décrivez une approche naïve pour distribuer le tableau et calculer la moyenne en parallèle en utilisant MPI. / 请描述一种朴素方法，使用 MPI 分发数组并并行计算平均值。

Correction : / 参考答案：

Approche naïve : distribuer les éléments du tableau de manière égale entre les processus, calculer la somme locale, puis utiliser `MPI_Reduce` pour obtenir la somme totale et calculer la moyenne. / 朴素方法：将数组元素平均分配给各个进程，各进程计算局部和，然后使用 `MPI_Reduce` 得到总和并计算平均值。

Quelle est la complexité de cette approche en termes de temps et de communication ? / 从时间和通信角度看，这种方法的复杂度是多少？

Correction : / 参考答案：

Complexité en temps : $O(n/p)$ pour le calcul local et $O(\log p)$ pour la réduction. / 时间复杂度：局部计算为 $O(n/p)$ ，归约为 $O(\log p)$ 。

Complexité en communication : $O(p)$ messages. / 通信复杂度： $O(p)$ 条消息。

Justification : chaque processus traite environ n/p éléments, donc la somme locale coûte $O(n/p)$. Ensuite, `MPI_Reduce` peut être vu comme une réduction en arbre : à chaque étape, le nombre de processus actifs est divisé par 2, donc il faut environ $\log_2 p$ étapes, d'où le coût $O(\log p)$. Si l'on compte le nombre total de messages échangés pendant la réduction, on obtient $p/2 + p/4 + \dots + 1 = p - 1$, donc $O(p)$. / 推导：每个进程大约处理 n/p 个元素，所以局部求和代价是 $O(n/p)$ 。随后，`MPI_Reduce` 可以看成树形归约：每一轮活跃进程数减半，因此大约需要 $\log_2 p$ 轮，所以归约时间为 $O(\log p)$ 。如果统计归约过程中交换的消息总数，则为 $p/2 + p/4 + \dots + 1 = p - 1$ ，因此通信复杂度是 $O(p)$ 。

2) Conception de l'algorithme (15 points) / 2) 算法设计 (15 分)

Proposez une méthode plus efficace pour distribuer le tableau et calculer la moyenne en parallèle. / 请提出一种更高效的方法来分发数组并并行计算平均值。

Correction : / 参考答案：

Méthode efficace : utiliser `MPI_Scatter` pour distribuer les éléments du tableau et `MPI_Reduce` pour obtenir la somme totale. / 更高效的方法：使用 `MPI_Scatter` 分发数组元素，使用 `MPI_Reduce` 求得总和。

Expliquez comment vous répartissez les tâches entre les processus. / 请说明如何在各进程之间分配任务。

Correction : / 参考答案：

Répartition des tâches : chaque processus reçoit une partie égale du tableau et calcule la somme locale. / 任务分配：每个进程接收数组的一部分并计算局部和。

Quelles communications collectives utiliser et pourquoi ? / 应使用哪些集合通信？为什么？

Correction : / 参考答案：

Communications collectives : `MPI_Scatter` pour la distribution et `MPI_Reduce` pour la réduction des sommes locales. / 集合通信：使用 `MPI_Scatter` 进行分发，使用 `MPI_Reduce` 归约各局部和。

3) Implémentation (25 points) / 3) 实现 (25 分)

Ecrire le pseudo-code de votre algorithme parallèle pour le calcul de la moyenne. / 写出你的并行平均值算法的伪代码。

Correction (pseudo-code) : / 参考答案 (伪代码)：

- Initialiser MPI et obtenir le rang et la taille du communicateur. / 初始化 MPI，并获取当前进程秩和 communicator 大小。
 - Si processus 0, lire le tableau et utiliser `MPI_Scatter` pour distribuer les éléments. / 若为 0 号进程，读取数组并使用 `MPI_Scatter` 分发元素。
 - Chaque processus calcule la somme locale de ses éléments. / 每个进程计算其局部元素之和。
 - Utiliser `MPI_Reduce` pour obtenir la somme totale sur le processus 0. / 使用 `MPI_Reduce` 在 0 号进程上得到总和。
 - Si processus 0, calculer la moyenne et afficher le résultat. / 若为 0 号进程，计算平均值并输出结果。
- Incluez les appels MPI nécessaires pour les communications. / 请给出通信所需的 MPI 调用。

Correction : / 参考答案：

Appels MPI : `MPI_Scatter`, `MPI_Reduce` / MPI 调用：`MPI_Scatter`、`MPI_Reduce`。

Décrivez les étapes clés de l'implémentation, y compris l'initialisation, la répartition des données, le calcul local de la moyenne, et la fusion des résultats. / 请描述实现中的关键步骤，包括初始化、数据分配、局部平均值计算以及结果合并。

Correction : / 参考答案：

- Initialisation MPI. / 初始化 MPI。
 - Distribution des données avec `MPI_Scatter`. / 使用 `MPI_Scatter` 分发数据。
 - Calcul local de la somme. / 本地计算部分和。
 - Réduction des sommes locales avec `MPI_Reduce`. / 使用 `MPI_Reduce` 归约局部和。
 - Calcul de la moyenne sur le processus 0. / 在 0 号进程上计算平均值。
- Ecrivez le code C complet pour l'implémentation. / 请写出该实现的完整 C 代码。

Correction (code complet) : / 参考答案 (完整代码)：

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int rank, size, n = 0;
    int *array = NULL, *local_array = NULL;
    int local_n, local_sum = 0, global_sum = 0;
    double average;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Entrez la taille du tableau : ");
        scanf("%d", &n);
        if (n % size != 0) {
            fprintf(stderr, "La taille du tableau doit etre un multiple de %d.\n", size);
        }
    }

    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (n % size != 0) {
        MPI_Finalize();
        return 1;
    }

    local_n = n / size;
    local_array = (int *)malloc(local_n * sizeof(int));

    if (rank == 0) {
        array = (int *)malloc(n * sizeof(int));
        printf("Entrez les éléments du tableau : ");
        for (int i = 0; i < n; i++) {
            scanf("%d", &array[i]);
        }
    }

    MPI_Scatter(array, local_n, MPI_INT, local_array, local_n, MPI_INT, 0, MPI_COMM_WORLD);

    for (int i = 0; i < local_n; i++) {
        local_sum += local_array[i];
    }

    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        average = (double)global_sum / n;
        printf("Somme globale : %d\n", global_sum);
        printf("Moyenne globale : %.6f\n", average);
        free(array);
    }

    free(local_array);
    MPI_Finalize();
    return 0;
}

```

Examen 2023-2024

Exercice 1 / 练习 1

Pour chacun des extraits de programme suivants, analyser le code, décrire le problème et proposer une solution. / 对于下面每一段程序片段，请分析代码，说明问题，并提出一种解决方案。

Question 1.1 / 问题 1

Dans ce qui suit, on désigne par `rang` le rang de la tâche MPI, et par `p` la taille du communicateur `MPI_COMM_WORLD`. / 在下面内容中，`rang` 表示 MPI 任务的秩，`P` 表示 communicator `MPI_COMM_WORLD` 的大小。

```

MPI_Request req;
MPI_Status sta1, sta2;
double m = 1.0 / (rang + 1);
int v = 2;
if (rang == 0 || rang == 2) {
    if (rang == 2) v = 0;
    MPI_Isend(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &req);
    MPI_Recv(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &sta1);
    MPI_Wait(&req, &sta2);
}

```

Correction / 参考答案

Problème : seuls les processus de rang 0 et 2 participent à l'échange. Le code réalise un échange croisé entre 0 et 2 avec `MPI_Isend` puis `MPI_Recv`. Cette écriture peut fonctionner, mais elle suppose implicitement que le communicateur contient au moins 3 processus. / 问题：只有秩 0 和秩 2 参与通信。该代码实现的是 0 和 2 之间的交叉交换：先 `MPI_Isend` 再 `MPI_Recv`。这种写法本身可以工作，但它隐含要求 communicator 中至少有 3 个进程。

Erreur principale : si $p < 3$, le processus 0 tente d'envoyer vers le rang 2, qui n'existe pas. / 主要错误：如果 $p < 3$ ，秩 0 会向不存在的秩 2 发送消息。

Solution : le plus petit correctif consiste à conserver le code presque inchangé et à vérifier que le rang 2 existe, c'est-à-dire $P > 2$. / 解决方案：最小修改方式是基本保持原代码不变，只补充检查秩 2 是否存在，也就是判断 $P > 2$ 。

Exemple corrigé : / 修正示例：

```

MPI_Request req;
MPI_Status sta1, sta2;
double m = 1.0 / (rang + 1);
int v = 2;
if (P > 2 && (rang == 0 || rang == 2)) {
    if (rang == 2) v = 0;
    MPI_Isend(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &req);
    MPI_Recv(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &sta1);
    MPI_Wait(&req, &sta2);
}

```

Idée : le code initial n'est pas faux dans son principe ; le vrai problème est qu'il suppose l'existence du processus 2. / 思路：原代码的通信思路本身不一定错；真正的问题是它默认进程 2 一定存在。

Question 1.2 / 问题 2

Dans ce qui suit, on désigne par `rang` le rang de la tâche MPI, et par `P` la taille du communicateur `MPI_COMM_WORLD`. / 在下面内容中，`rang` 表示 MPI 任务的秩，`P` 表示 communicator `MPI_COMM_WORLD` 的大小。

```

MPI_Status sta;
int l, r, w;
w = rang;
l = (rang + P - 1) % P;
r = (rang + 1) % P;
MPI_Send(&w, 1, MPI_INT, l, 0, MPI_COMM_WORLD);
MPI_Recv(&w, 1, MPI_INT, r, 0, MPI_COMM_WORLD, &sta);

```

Correction / 参考答案

Problème : tous les processus exécutent `MPI_Send` puis `MPI_Recv` dans le même ordre ; avec des communications bloquantes, cela peut provoquer un interblocage (deadlock). / 问题：所有进程都按相同顺序执行 `MPI_Send` 再 `MPI_Recv`；在阻塞通信下，这可能导致死锁。

Solution : utiliser des communications non bloquantes (`MPI_Isend` / `MPI_Irecv`), puis attendre leur terminaison. / 解决方案：使用非阻塞通信 (`MPI_Isend` / `MPI_Irecv`)，然后等待通信完成。

```

MPI_Request req1, req2;
MPI_Status sta;
int l, r, w;
w = rang;
l = (rang + P - 1) % P;
r = (rang + 1) % P;

MPI_Isend(&w, 1, MPI_INT, l, 0, MPI_COMM_WORLD, &req1);
MPI_Irecv(&w, 1, MPI_INT, r, 0, MPI_COMM_WORLD, &req2);
MPI_Wait(&req1, &sta);
MPI_Wait(&req2, &sta);

```

Résultat attendu : chaque processus reçoit la valeur du voisin de droite dans l'anneau. / 预期结果：每个进程最终收到其右邻居发送的值。

Question 1.3 / 问题 3

Dans ce qui suit, on désigne par `rang` le rang de la tâche MPI, et par `P` la taille du communicateur `MPI_COMM_WORLD`. / 在下面内容中，`rang` 表示 MPI 任务的秩，`P` 表示 communicator `MPI_COMM_WORLD` 的大小。

```
double som_glob, som_loc = 0;
/* x est un tableau distribué de réels (double précision)
   N est le nombre d'éléments associé à la tâche
*/
for (i = 0; i < N; i++) {
    som_loc += x[i];
}
MPI_Reduce(&som_loc, &som_glob, 1, MPI_DOUBLE, MPI_SUM, rang, MPI_COMM_WORLD);
if (rang == 0) {
    printf("Somme globale: %g\n", som_glob);
}
```

Correction / 参考答案

Problème : l'appel à `MPI_Reduce` utilise `rang` comme racine. Or la racine doit être la même sur tous les processus. Ici, chaque processus fournit une valeur différente (0, 1, 2, ...), ce qui rend l'appel incorrect. / 问题： `MPI_Reduce` 把 `rang` 当作根进程，但根进程参数必须在所有进程上保持一致。这里每个进程传入的根都不同（0、1、2.....），因此调用是错误的。

Solution : choisir une racine unique, par exemple 0. / 解决方案：选择唯一的根进程，例如 0。

```
double som_glob, som_loc = 0.0;
for (i = 0; i < N; i++) {
    som_loc += x[i];
}
MPI_Reduce(&som_loc, &som_glob, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
if (rang == 0) {
    printf("Somme globale: %g\n", som_glob);
}
```

Explication : chaque processus calcule sa somme locale, puis `MPI_Reduce` additionne toutes les sommes locales et place le résultat final sur le processus de rang 0. / 说明：每个进程先计算自己的局部和，再由 `MPI_Reduce` 把所有局部和相加，并将最终结果放到秩 0 上。

Exercice 2 / 练习 2

Proposer un algorithme réalisant un échange de messages MPI sous la forme d'un anneau via des communications de type `MPI_Send / MPI_Recv`. / 设计一个算法，使用 `MPI_Send / MPI_Recv` 类型的通信，实现一个环形 MPI 消息传递。

Correction / 参考答案

Principe : chaque processus envoie un message à son voisin de droite et reçoit un message de son voisin de gauche. / 原理：每个进程向右邻居发送一条消息，并从左邻居接收一条消息。

Définition des voisins : / 邻居定义：

- voisin gauche : $(rang - 1 + P) \% P$ / 左邻居： $(rang - 1 + P) \% P$
- voisin droit : $(rang + 1) \% P$ / 右邻居： $(rang + 1) \% P$

Pour éviter l'interblocage avec `MPI_Send / MPI_Recv`, on casse la symétrie : les rangs pairs envoient puis reçoivent, les rangs impairs reçoivent puis envoient. / 为避免 `MPI_Send / MPI_Recv` 的死锁，需要打破对称性：偶数秩先发送后接收，奇数秩先接收后发送。

Pseudo-code : / 伪代码：

```
int gauche = (rang - 1 + P) % P;
int droite = (rang + 1) % P;
int send_val = rang;
int recv_val;

if (rang % 2 == 0) {
    MPI_Send(&send_val, 1, MPI_INT, droite, 0, MPI_COMM_WORLD);
    MPI_Recv(&recv_val, 1, MPI_INT, gauche, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
} else {
    MPI_Recv(&recv_val, 1, MPI_INT, gauche, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    MPI_Send(&send_val, 1, MPI_INT, droite, 0, MPI_COMM_WORLD);
}
```

Complexité : chaque processus effectue un envoi et une réception, donc le coût local est constant, soit $O(1)$, et le nombre total de messages est P . / 复杂度：每个进程执行一次发送和一次接收，因此局部代价是常数级 $O(1)$ ，总消息数为 P 。

Exercice 3 / 练习 3

Le programme suivant permet de calculer un produit scalaire entre deux vecteurs de taille `N`. / 下面的程序用于计算两个大小为 `N` 的向量的点积。

Cette petite application est séquentielle. Le but de cet exercice est de paralléliser ce code avec plusieurs modèles de programmation parallèle. / 该小程序是顺序版本。本题的目标是使用多种并行编程模型对其进行并行化。

```
double *x;
double *y;
int N;
double final_res = 0.0;

double compute(double *x, double *y, int N) {
    double res = 0.0;
    int i;
    for (i = 0; i < N; i++)
        res += x[i] * y[i];
    return res;
}

int main(int argc, char **argv) {
    /* La fonction initialise_N permet d'initialiser la valeur de N */
    N = initialise_N();
    /* Allocation des vecteurs x et y de taille N */
    x = (double *)malloc(N * sizeof(double));
    y = (double *)malloc(N * sizeof(double));
    /* Initialisation des vecteurs x et y */
    /* La fonction readVectors remplit les tableaux x et y */
    readVectors(x, y, N);
    /* Calcul du produit scalaire */
    final_res = compute(x, y, N);
    printf("Dot product with %d coordinates: %g\n", N, final_res);
}
```

Question 3.1 / 问题 1

Décrire brièvement où se trouve le parallélisme potentiel dans l'application séquentielle ci-dessus. / 请简要说明上述顺序程序中的潜在并行性在哪里。

Correction / 参考答案

Le parallélisme potentiel se trouve dans la boucle de calcul du produit scalaire. / 潜在并行性位于点积计算循环中。

En effet, chaque itération calcule indépendamment le produit $x[i] * y[i]$, puis l'ajoute à une somme. / 因为每次迭代都独立计算 $x[i] * y[i]$ ，然后加入总和。

On peut donc : / 因此可以：

- répartir les indices du tableau entre plusieurs processus MPI ; / 把数组下标分配给多个 MPI 进程；
- calculer un résultat partiel sur chaque processus ; / 每个进程计算一个局部结果；
- effectuer ensuite une réduction pour obtenir le produit scalaire global. / 然后通过归约得到全局点积结果。

Question 3.2 / 问题 2

Ecrire la fonction `main` avec un parallélisme à mémoire distribuée avec le modèle MPI / 使用 MPI 的分布式内存并行模型编写函数 `main`

(il est interdit de modifier les appels aux fonctions `initialise_N` et `readVectors`). / (禁止修改对函数 `initialise_N` 和 `readVectors` 的调用)。

Attention, ne pas oublier les étapes suivantes : / 注意，不要遗漏以下步骤：

- Initialisation / terminaison du modèle MPI (on admettra que `N` est un multiple du nombre de tâches MPI). / MPI 模型的初始化与结束（可假设 `N` 是 MPI 任务数的整数倍）。
- Distribution des tableaux `x` et `y` sur les différentes tâches MPI après initialisation des tableaux avec la fonction `readVectors` appelée par une seule tâche MPI. / 在仅由一个 MPI 任务调用 `readVectors` 初始化 `x` 和 `y` 后，将这两个数组分发到不同的 MPI 任务上。
- Affichage du résultat final sur la tâche de rang 0. / 在秩为 0 的任务上输出最终结果。

Correction / 参考答案

Idée : le processus 0 initialise `N`, alloue et remplit les vecteurs complets, puis distribue des blocs de taille `N / P` aux autres processus avec `MPI_Scatter`. Chaque processus calcule son produit scalaire local, puis `MPI_Reduce` permet d'obtenir le résultat final sur le rang 0. / 思路：由进程 0 初始化 `N`，分配并填充完整向量，然后用 `MPI_Scatter` 把大小为 `N / P` 的分块分发给其他进程。每个进程计算自己的局部点积，再通过 `MPI_Reduce` 在秩 0 上得到最终结果。

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

double *x;
double *y;
int N;
double final_res = 0.0;

double compute(double *x, double *y, int N) {
    double res = 0.0;
    int i;
    for (i = 0; i < N; i++)
        res += x[i] * y[i];
    return res;
}

int main(int argc, char **argv) {
    int rang, P;
    int local_N;
    double *local_x, *local_y;
    double local_res = 0.0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    if (rang == 0) {
        N = initialise_N();
    }

    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);

    local_N = N / P;
    local_x = (double *)malloc(local_N * sizeof(double));
    local_y = (double *)malloc(local_N * sizeof(double));

    if (rang == 0) {
        x = (double *)malloc(N * sizeof(double));
        y = (double *)malloc(N * sizeof(double));
        readVectors(x, y, N);
    }

    MPI_Scatter(x, local_N, MPI_DOUBLE, local_x, local_N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Scatter(y, local_N, MPI_DOUBLE, local_y, local_N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    local_res = compute(local_x, local_y, local_N);

    MPI_Reduce(&local_res, &final_res, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rang == 0) {
        printf("Dot product with %d coordinates: %g\n", N, final_res);
        free(x);
        free(y);
    }

    free(local_x);
    free(local_y);

    MPI_Finalize();
    return 0;
}

```

Explication : / 说明:

- **MPI_Bcast** diffuse **N** à tous les processus. / **MPI_Bcast** 将 **N** 广播给所有进程。
- **readVectors** n'est appelée que par le processus 0, comme demandé. / 按要求, **readVectors** 只由进程 0 调用。
- **MPI_Scatter** distribue les morceaux de **x** et **y**. / **MPI_Scatter** 分发 **x** 和 **y** 的各个分块。
- **compute** est utilisée localement sur chaque tâche MPI. / **compute** 在每个 MPI 任务上本地计算。
- **MPI_Reduce** additionne les résultats partiels pour obtenir le produit scalaire global. / **MPI_Reduce** 将局部结果相加, 得到全局点积。